

Python Coding Round

Common Coding Problems with Solutions

55 questions with answers

Interview Preparation Guide

Coding Problems & Solutions

Q1. Reverse a string

Write a function to reverse a string without using built-in reverse().

```
def reverse_string(s):
    result = ''
    for ch in s:
        result = ch + result
    return result

# Pythonic one-liner:
def reverse_string(s):
    return s[::-1]
```

Q2. Check palindrome

Determine whether a given string is a palindrome (ignoring case).

```
def is_palindrome(s):
    s = s.lower()
    return s == s[::-1]
```

Q3. Check anagram

Check if two strings are anagrams of each other.

```
def is_anagram(a, b):
    return sorted(a.lower()) == sorted(b.lower())

# O(n) with Counter:
from collections import Counter
def is_anagram(a, b):
    return Counter(a) == Counter(b)
```

Q4. FizzBuzz

Print numbers 1..n, but 'Fizz' for multiples of 3, 'Buzz' for 5, 'FizzBuzz' for both.

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0:
            print('FizzBuzz')
        elif i % 3 == 0:
            print('Fizz')
        elif i % 5 == 0:
            print('Buzz')
        else:
            print(i)
```

Q5. Factorial (iterative and recursive)

Compute the factorial of n.

```
def factorial_iter(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def factorial_rec(n):
    return 1 if n <= 1 else n * factorial_rec(n - 1)
```

Q6. Fibonacci sequence

Generate the first n Fibonacci numbers efficiently.

```
def fibonacci(n):
    a, b = 0, 1
    result = []
    for _ in range(n):
        result.append(a)
        a, b = b, a + b
    return result
```

Q7. Check prime number

Determine whether a number is prime.

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```

Q8. Count vowels and consonants

Count vowels and consonants in a string.

```
def count_vc(s):
    vowels = set('aeiou')
    v = c = 0
    for ch in s.lower():
        if ch.isalpha():
            if ch in vowels:
                v += 1
            else:
                c += 1
    return v, c
```

Q9. Find the largest element in a list

Return the maximum value without using max().

```
def find_max(nums):
    largest = nums[0]
    for n in nums[1:]:
        if n > largest:
            largest = n
    return largest
```

Q10. Second largest element

Find the second largest element in a list.

```
def second_largest(nums):
    first = second = float('-inf')
    for n in nums:
        if n > first:
            first, second = n, first
        elif first > n > second:
            second = n
    return second
```

Q11. Remove duplicates from a list

Remove duplicates while preserving order.

```
def dedupe(nums):
    seen = set()
    result = []
    for n in nums:
        if n not in seen:
            seen.add(n)
            result.append(n)
    return result
```

Q12. Count word frequency

Count the frequency of each word in a sentence.

```
from collections import Counter
def word_freq(text):
    return dict(Counter(text.lower().split()))
```

Q13. First non-repeating character

Return the first character that appears only once.

```
from collections import Counter
def first_unique(s):
    counts = Counter(s)
    for ch in s:
        if counts[ch] == 1:
```

```
        return ch
    return None
```

Q14. Two Sum

Return indices of two numbers that add up to a target.

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []
```

Q15. Reverse a list in place

Reverse a list without slicing or reversed().

```
def reverse_list(lst):
    i, j = 0, len(lst) - 1
    while i < j:
        lst[i], lst[j] = lst[j], lst[i]
        i += 1
        j -= 1
    return lst
```

Q16. Merge two sorted lists

Merge two sorted lists into one sorted list.

```
def merge(a, b):
    i = j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
```

Q17. Binary search

Implement binary search on a sorted list.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
```

```

        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

```

Q18. Bubble sort

Sort a list using bubble sort.

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

```

Q19. Quick sort

Sort a list using quicksort.

```

def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

```

Q20. Merge sort

Sort a list using merge sort.

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    i = j = 0
    out = []
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1

```

```
out.extend(left[i:]); out.extend(right[j:])
return out
```

Q21. Sum of digits

Compute the sum of digits of an integer.

```
def digit_sum(n):
    n = abs(n)
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total
```

Q22. Reverse an integer

Reverse the digits of an integer.

```
def reverse_int(n):
    sign = -1 if n < 0 else 1
    rev = int(str(abs(n))[::-1])
    return sign * rev
```

Q23. Check Armstrong number

Check if a number equals the sum of its digits each raised to the count of digits.

```
def is_armstrong(n):
    digits = str(n)
    p = len(digits)
    return n == sum(int(d) ** p for d in digits)
```

Q24. GCD and LCM

Compute the GCD and LCM of two numbers.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)
```

Q25. Count occurrences of an element

Count how many times a value appears in a list.

```
def count_occurrences(lst, target):
    count = 0
    for x in lst:
        if x == target:
            count += 1
```

```
return count
```

Q26. Flatten a nested list

Flatten an arbitrarily nested list into a flat list.

```
def flatten(nested):
    result = []
    for item in nested:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result
```

Q27. Find missing number 1..n

Given n-1 distinct numbers from 1..n, find the missing one.

```
def missing_number(nums, n):
    expected = n * (n + 1) // 2
    return expected - sum(nums)
```

Q28. Find duplicates in a list

Return all elements that appear more than once.

```
from collections import Counter
def find_duplicates(nums):
    return [k for k, v in Counter(nums).items() if v > 1]
```

Q29. Move all zeros to end

Move all zeros in a list to the end while keeping order of non-zeros.

```
def move_zeros(nums):
    pos = 0
    for n in nums:
        if n != 0:
            nums[pos] = n
            pos += 1
    for i in range(pos, len(nums)):
        nums[i] = 0
    return nums
```

Q30. Rotate a list by k

Rotate a list to the right by k positions.

```
def rotate(nums, k):
    k %= len(nums)
    return nums[-k:] + nums[:-k]
```

Q31. Check balanced parentheses

Check whether brackets `()[]{}` in a string are balanced.

```
def is_balanced(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

Q32. Longest common prefix

Find the longest common prefix among a list of strings.

```
def longest_common_prefix(strs):
    if not strs:
        return ''
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
            if not prefix:
                return ''
    return prefix
```

Q33. Maximum subarray sum (Kadane)

Find the contiguous subarray with the largest sum.

```
def max_subarray(nums):
    best = current = nums[0]
    for n in nums[1:]:
        current = max(n, current + n)
        best = max(best, current)
    return best
```

Q34. Count set bits

Count the number of 1s in the binary representation of n.

```
def count_bits(n):
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count
```

Q35. Find pairs with given sum

Find all unique pairs in a list that sum to a target.

```
def pair_sum(nums, target):
    seen = set()
    pairs = set()
    for n in nums:
        complement = target - n
        if complement in seen:
            pairs.add(tuple(sorted((n, complement))))
        seen.add(n)
    return list(pairs)
```

Q36. Title case a sentence

Capitalize the first letter of each word.

```
def title_case(text):
    return ' '.join(w.capitalize() for w in text.split())
```

Q37. Find the longest word

Return the longest word in a sentence.

```
def longest_word(text):
    return max(text.split(), key=len)
```

Q38. Sum of even numbers

Return the sum of even numbers in a list.

```
def sum_even(nums):
    return sum(n for n in nums if n % 2 == 0)
```

Q39. Transpose a matrix

Transpose a 2D matrix (list of lists).

```
def transpose(matrix):
    return [list(row) for row in zip(*matrix)]
```

Q40. Merge two dictionaries

Merge two dicts; values from the second override the first.

```
def merge_dicts(a, b):
    return {**a, **b}
```

Q41. Group anagrams

Group a list of words into anagram clusters.

```
from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[''.join(sorted(w))].append(w)
    return list(groups.values())
```

Q42. Decimal to binary

Convert a non-negative integer to its binary string.

```
def to_binary(n):
    if n == 0:
        return '0'
    bits = ''
    while n > 0:
        bits = str(n % 2) + bits
        n //= 2
    return bits
```

Q43. Check perfect number

A perfect number equals the sum of its proper divisors.

```
def is_perfect(n):
    return n > 0 and sum(i for i in range(1, n) if n % i == 0) == n
```

Q44. Find common elements in two lists

Return elements present in both lists.

```
def common_elements(a, b):
    return list(set(a) & set(b))
```

Q45. Caesar cipher

Shift each letter by k positions (encrypt).

```
def caesar(text, k):
    result = []
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result.append(chr((ord(ch) - base + k) % 26 + base))
        else:
            result.append(ch)
    return ''.join(result)
```

Q46. Count words, chars, lines in text

Return counts of lines, words, and characters in a block of text.

```
def text_stats(text):
    lines = text.splitlines()
    words = text.split()
    return {'lines': len(lines), 'words': len(words), 'chars': len(text)}
```

Q47. Find intersection node count / unique counts

Return how many distinct values appear in a list.

```
def distinct_count(nums):
    return len(set(nums))
```

Q48. Implement a stack using a list

Implement a basic stack with push, pop, peek.

```
class Stack:
    def __init__(self):
        self._data = []
    def push(self, x):
        self._data.append(x)
    def pop(self):
        return self._data.pop()
    def peek(self):
        return self._data[-1]
    def is_empty(self):
        return not self._data
```

Q49. Implement a queue using two stacks

Build a FIFO queue using two stacks.

```
class Queue:
    def __init__(self):
        self.inbox, self.outbox = [], []
    def enqueue(self, x):
        self.inbox.append(x)
    def dequeue(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        return self.outbox.pop()
```

Q50. Sum of list using recursion

Compute the sum of a list recursively.

```
def rec_sum(nums):
    if not nums:
        return 0
```

```
return nums[0] + rec_sum(nums[1:])
```

Q51. Power of a number

Compute a raised to b using fast exponentiation.

```
def power(a, b):
    result = 1
    while b > 0:
        if b & 1:
            result *= a
        a *= a
        b >>= 1
    return result
```

Q52. Find the median of a list

Return the median value of a list of numbers.

```
def median(nums):
    s = sorted(nums)
    n = len(s)
    mid = n // 2
    if n % 2:
        return s[mid]
    return (s[mid - 1] + s[mid]) / 2
```

Q53. Remove punctuation from text

Strip all punctuation characters from a string.

```
import string
def remove_punctuation(text):
    return text.translate(str.maketrans('', '', string.punctuation))
```

Q54. Check if two strings are rotations

Check whether one string is a rotation of another.

```
def is_rotation(a, b):
    return len(a) == len(b) and b in (a + a)
```

Q55. Find the longest substring without repeats

Length of the longest substring without repeating characters.

```
def longest_unique(s):
    seen = {}
    start = best = 0
    for i, ch in enumerate(s):
        if ch in seen and seen[ch] >= start:
            start = seen[ch] + 1
        seen[ch] = i
        best = max(best, i - start + 1)
```

```
return best
```